

# Predictor-Feedback Stabilization of Linear Switched Systems with State-Dependent Switching or Delayed Switching Input

Andreas Katsanikakis, Nikolaos Bekiaris-Liberis, and Delphine Bresch-Pietri,\*

## 1 Purpose of this Document

This document describes the MATLAB code used to reproduce the numerical examples in Section V of the paper “*Predictor-Feedback Stabilization of Linear Switched Systems with State-Dependent Switching or Delayed Switching Input*” by Andreas Katsanikakis, Nikolaos Bekiaris-Liberis, and Delphine Bresch-Pietri.

The preprint is available at <https://hal.science/hal-05579119>.

The document explains:

- the system setup and the main equations implemented in the scripts,
- the numerical implementation of the two predictor-feedback designs,
- the BMI/LMI search used in the second numerical example,
- the structure of the repository and the role of each MATLAB file,
- how to run the simulations and reproduce the corresponding plots that appear in the paper.

## 2 Simulation Example V.A: State-Dependent Switching

### 2.1 System Setup

The first numerical example illustrates Theorem 3.1. The plant is the state-dependent switched system with delayed control input

$$\dot{X}(t) = A_{\sigma(X(t))}X(t) + B_{\sigma(X(t))}U(t - D), \quad (1)$$

with the two-mode numerical data

$$\begin{aligned} A_1 &= \begin{bmatrix} 1 & -5 \\ 0 & 3 \end{bmatrix}, & B_1 &= \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \\ A_2 &= \begin{bmatrix} 3 & 0 \\ 5 & 1 \end{bmatrix}, & B_2 &= \begin{bmatrix} 1 \\ 0 \end{bmatrix}. \end{aligned} \quad (64)$$

The feedback gains and local Lyapunov matrices used in the example are

$$\begin{aligned} S_1 &= \begin{bmatrix} 1.0432 & 1.0031 \\ 1.0031 & 2.9722 \end{bmatrix}, & K_1 &= \begin{bmatrix} 0 & -2 \end{bmatrix}, \\ S_2 &= \begin{bmatrix} 2.9722 & -1.0031 \\ -1.0031 & 1.0432 \end{bmatrix}, & K_2 &= \begin{bmatrix} -2 & 0 \end{bmatrix}. \end{aligned} \quad (65)$$

---

\*Funded by the European Union (ERC, C-NORA, 101088147). Views and opinions expressed are however those of the authors only and do not necessarily reflect those of the European Union or the European Research Council Executive Agency. Neither the European Union nor the granting authority can be held responsible for them.

Define

$$E_i(X) = X^\top S_i X, \quad i = 1, 2. \quad (66)$$

The switching law used in the simulation is

$$\sigma(X) = \max \left\{ \arg \max_{i \in \{1,2\}} E_i(X) \right\}, \quad (67)$$

so that

$$\begin{aligned} \Omega_1 &= \{X : E_1(X) > E_2(X)\}, \\ \Omega_2 &= \{X : E_1(X) \leq E_2(X)\}. \end{aligned} \quad (68)$$

The outer maximum in (67) provides the deterministic tie-breaking rule implemented in the local MATLAB function `sigma_paper` in `Sim_IV_A.m`: equality is assigned to mode 2.

## 2.2 Predictor-Feedback Law

The predictor state is defined implicitly over one delay interval by

$$P(\theta) = X(t) + \int_{t-D}^{\theta} [A_{\sigma(P(s))}P(s) + B_{\sigma(P(s))}U(s)] ds, \quad t - D \leq \theta \leq t, \quad (8)$$

and the predictor-feedback law is

$$U(t) = K_{\sigma(P(t))}P(t). \quad (9)$$

Because the switching mode depends on the predictor state itself, the predictor in (8) is implicit. In `Sim_IV_A.m`, the implicit predictor is computed inside the main simulation loop, in the inner loop for `j = 1:N`. Starting from the current plant state,

$$P_0 = X(t),$$

the predictor is propagated over the interval  $[t - D, t]$  using the left-endpoint Euler rule

$$P_{j+1} = P_j + h \left( A_{\sigma(P_j)}P_j + B_{\sigma(P_j)}U_j \right),$$

where  $h = \Delta t$  and  $U_j$  denotes the stored control value at the corresponding point. After each Euler step, the state-dependent switching law (67) is recomputed using the newly computed predictor state:

$$\sigma(P_{j+1}) = \begin{cases} 1, & P_{j+1}^\top S_1 P_{j+1} > P_{j+1}^\top S_2 P_{j+1}, \\ 2, & P_{j+1}^\top S_1 P_{j+1} \leq P_{j+1}^\top S_2 P_{j+1}. \end{cases}$$

This corresponds to the variables `P_prev`, `P_next`, and `modeP` in the code. At the end of the prediction horizon, the computed value is used as  $P(t)$  in the feedback law (9).

## 2.3 Numerical Implementation

The simulation uses

- delay  $D = 2$  s,
- initial state  $X(0) = [2 \quad -1]^\top$ ,
- initial control history  $U(s) = 0$  for  $s \in [-D, 0)$ ,
- final time  $T = 10$  s,
- simulation and predictor step sizes  $\Delta t = h = 10^{-3}$  s.

The script `Sim_IV_A.m` performs the simulation and saves the workspace in `myworkspace_IV_A.mat`. The script `plots_IV_A.m` loads this file and generates the state and control-input plots.

### 3 Simulation Example V.B: Delayed Switching Input

#### 3.1 System Setup

The second numerical example illustrates Theorem 4.3. In this case, the switching signal is itself the control input and acts on the plant after a constant delay  $D$ . The system is

$$\dot{X}(t) = A_{U(t-D)}X(t). \quad (42)$$

The numerical example uses  $D = 2$  s and the two unstable modes

$$A_1 = \begin{bmatrix} 1 & -5 \\ 0 & 1 \end{bmatrix}, \quad A_2 = \begin{bmatrix} 1 & 0 \\ 5 & 1 \end{bmatrix}. \quad (69)$$

The matrices used to define the nominal state-dependent switching law are

$$\begin{aligned} S_1 &= \begin{bmatrix} 3.2487 & 6.9864 \\ 6.9864 & 31.3624 \end{bmatrix}, & Q_1 &= \begin{bmatrix} 0.1503 & -0.4195 \\ -0.4195 & 1.6480 \end{bmatrix}, \\ S_2 &= \begin{bmatrix} 46.1415 & -12.6366 \\ -12.6366 & 7.1167 \end{bmatrix}, & Q_2 &= \begin{bmatrix} 3.5098 & 1.0354 \\ 1.0354 & 0.5323 \end{bmatrix}. \end{aligned} \quad (70)$$

These matrices satisfy Assumption 4.1. In the repository, they are obtained using the auxiliary script `find_LMI_IV_B.m` and are then specified directly in `Sim_IV_B.m` for the simulation.

#### 3.2 BMI/LMI Search

The auxiliary script `find_LMI_IV_B.m` performs the search used to obtain the matrices in (70). For each pair of modes  $(i, j)$ , the regional Lyapunov decrease condition in equation (43) of the paper requires

$$X^\top \left( A_i^\top S_i + S_i A_i + Q_i \right) X \leq 0 \quad \text{whenever} \quad X^\top (S_i - S_j) X \geq 0. \quad (1)$$

The MATLAB implementation utilizes the S-procedure logic. For the two-mode case, the resulting sufficient conditions, under the S-procedure, are

$$\begin{aligned} A_1^\top S_1 + S_1 A_1 + Q_1 + \gamma_{12}(S_1 - S_2) &\prec 0, \\ A_2^\top S_2 + S_2 A_2 + Q_2 + \gamma_{21}(S_2 - S_1) &\prec 0. \end{aligned} \quad (2)$$

In `find_LMI_IV_B.m`, the multipliers  $\gamma_{12}$  and  $\gamma_{21}$  correspond to the MATLAB variables `g12` and `g21`. They are scanned over the logarithmic grid defined by

```
gammas = logspace(-4,4,41);
```

For each fixed pair  $(g12, g21)$ , the matrices `S1`, `S2`, `Q1`, and `Q2` are decision variables of a semidefinite feasibility problem solved using CVX. Since the multipliers are fixed at each grid point, each individual CVX problem becomes an LMI feasibility problem.

The no-sliding conditions in equations (44),(45) are then checked for both pairs  $(1, 2)$  and  $(2, 1)$ . These conditions are imposed on the pairwise equality set

$$\Gamma_{ij} = \{X : X^\top (S_i - S_j) X = 0\}.$$

In the code, these tests are performed by the local MATLAB function `check_sliding_pair`. For a given pair  $(i, j)$ , the function checks the two alternative LMI conditions corresponding to equations (44) and (45) and returns `true` when one of them (at least) is feasible.

Once both the regional Lyapunov decrease tests and the no-sliding tests are satisfied, the script prints the resulting matrices  $S_1, S_2, Q_1, Q_2$ . These are the matrices reported in (70) and used in `Sim_IV_B.m`.

### 3.3 Predictor-Feedback Switching Law

Define

$$E_i(X) = X^\top S_i X. \quad (46)$$

The nominal switching law is

$$\sigma(X) = \max \left\{ \arg \max_{i \in \{1,2\}} E_i(X) \right\}, \quad (47)$$

and the predictor-feedback switching law is

$$U(t) = \sigma(P(t)). \quad (49)$$

For the two-mode case, the local MATLAB function `sigma_paper` in `Sim_IV_B.m` implements (47) as

$$\sigma(x) = \begin{cases} 1, & x^\top S_1 x > x^\top S_2 x, \\ 2, & x^\top S_1 x \leq x^\top S_2 x. \end{cases}$$

Thus, equality is assigned to mode 2, aligned with the outer maximum in (47). The predictor is then defined on  $[t - D, t]$  by

$$\dot{P}(\theta) = A_{U(\theta)} P(\theta), \quad P(t - D) = X(t). \quad (50)$$

Let

$$t - D = \tau_0 < \tau_1 < \dots < \tau_k < \tau_{k+1} = t$$

be the switching instants of the stored input history over the delay interval, with  $U(\theta) = m_i$  on  $(\tau_i, \tau_{i+1})$ . The predictor can then be written explicitly as

$$P(t) = e^{A_{m_k}(t-\tau_k)} \dots e^{A_{m_0}(\tau_1-t+D)} X(t). \quad (52)$$

In the two-mode example, the corresponding switching regions are

$$\begin{aligned} \Omega_1 &= \{X : X^\top (S_1 - S_2) X > 0\}, \\ \Omega_2 &= \{X : X^\top (S_2 - S_1) X \geq 0\}. \end{aligned} \quad (71)$$

In contrast with the implicit predictor used in Example V.A, the predictor in (52) is explicit. At time  $t$ , the switching-input history on  $[t - D, t)$  is already known, so the switching instants and the corresponding active modes needed to construct the product of matrix exponentials are available directly.

## 4 Repository Structure

The repository contains:

- `Sim_IV_A.m` simulation of the numerical example in Section V.A,
- `plots_IV_A.m` state and control-input plots for Section V.A,
- `find_LMI_IV_B.m` auxiliary BMI/LMI search for the matrices used in Section V.B,
- `Sim_IV_B.m` simulation of the numerical example in Section V.B,
- `plots_IV_B.m` state and switching-input plots for Section V.B,
- `READMEcod.pdf` detailed documentation of the MATLAB implementation,
- `README.md` repository overview and brief instructions,
- `LICENSE.txt` licensing information.

## 5 How to Run the Simulations

### Requirements

- MATLAB,
- CVX toolbox for `find_LMI_IV_B.m`.

CVX is not required to run the two main simulation scripts because the matrices used in Section V.B are specified directly in `Sim_IV_B.m`.

### Section V.A

Run

```
Sim_IV_A.m  
plots_IV_A.m
```

The first script generates `myworkspace_IV_A.mat` and the second script loads the saved workspace and produces the plots.

### Section V.B

Run

```
Sim_IV_B.m  
plots_IV_B.m
```

The first script generates `myworkspace_IV_B.mat` and the second script loads the saved workspace and produces the plots.

To reproduce the BMI/LMI search used to obtain the matrices in equation (70), run

```
find_LMI_IV_B.m
```

with CVX toolbox correctly installed.

## References

- [1] A. Katsanikakis, N. Bekiaris-Liberis, and D. Bresch-Pietri, “Predictor-Feedback Stabilization of Linear Switched Systems with State-Dependent Switching or Delayed Switching Input,” preprint, 2026. <https://hal.science/hal-05579119>.